| EXP:2<br>8/7/2026 | PaaS in Public Cloud Using Render |
|---|---|

## AIM

To develop and deploy a simple web application using Render, a Platform as a Service (PaaS), and understand how a cloud platform hosts an application without requiring the user to manage the underlying servers.

## REQUIREMENTS

- Computer with Internet connection
- Python 3.10 or later
- Git / GitHub account
- GitHub repository
- Render account

## APPLICATION USED

A simple Python Flask web application was created with two routes: the Home page (/) and the About page (/about). The application was stored in GitHub and deployed as a Web Service on Render.

## PROJECT FILES

Project structure:

```
RenderHelloApp/
├── app.py
└── requirements.txt
```

Contents of app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! Welcome to Render PaaS."

@app.route("/about")
def about():
    return "This application is deployed on Render."
```

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Contents of requirements.txt:

```
Flask==3.0.3
gunicorn==22.0.0
```

# PROCEDURE

1. Create a project folder named RenderHelloApp.
2. Create app.py and requirements.txt inside the project folder and enter the required Flask application code and dependencies.
3. Create a public GitHub repository named RenderHelloApp.
4. Upload app.py and requirements.txt to the GitHub repository and commit the changes.
5. Open Render and create a new Web Service.
6. Connect the GitHub account and select the RenderHelloApp repository.
7. Configure the service with Python as the language/environment and main as the branch.
8. Leave the Root Directory blank.
9. Set the Build Command to: pip install -r requirements.txt
10. Set the Start Command to: gunicorn app:app
11. Select the Free compute instance when available.
12. Deploy the Web Service.
13. Wait until the Render dashboard displays Deploy succeeded / Live.
14. Open the generated public URL and verify the Home page.
15. Open the /about route and verify the About page.

# COMMANDS / CONFIGURATION USED

Build command:

```
pip install -r requirements.txt
```

Start command:

```
gunicorn app:app
```

Application routes:

```
https://renderhelloapp-d7i6.onrender.com/
https://renderhelloapp-d7i6.onrender.com/about
```

# OUTPUT

After successful deployment, Render displayed the service as live.

| Test Case | URL | Expected / Obtained Output |
|---|---|---|
| Render Deployment | Render Dashboard | Deploy succeeded \| Live |

| | | |
|---|---|---|
| Test Case 1 – Home Page | https://renderhelloapp-d7i6.onrender.com/ | Hello! Welcome to Render PaaS. |
| Test Case 2 – About Page | https://renderhelloapp-d7i6.onrender.com/about | This application is deployed on Render. |

# SCREENSHOTS / EVIDENCE

Attach the following screenshots in the record:

- Screenshot 1 – Render Dashboard showing "Deploy succeeded | Live" and the public URL.
- Screenshot 2 – Home page showing "Hello! Welcome to Render PaaS."
- Screenshot 3 – About page showing "This application is deployed on Render."

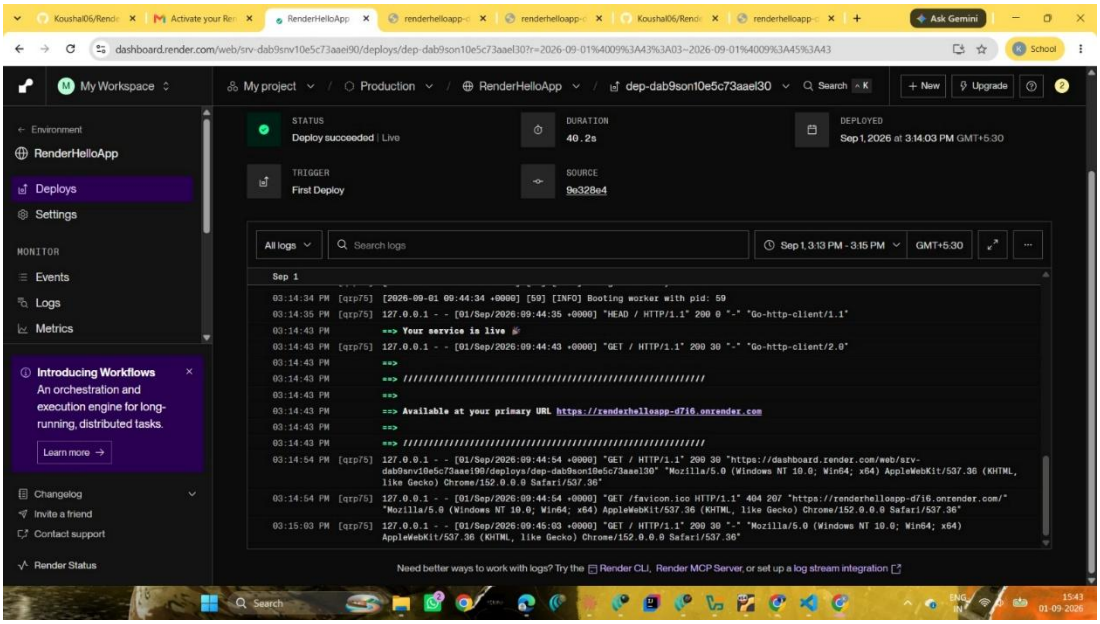



**Fig:Render Dashboard – Deployment successful / Live**

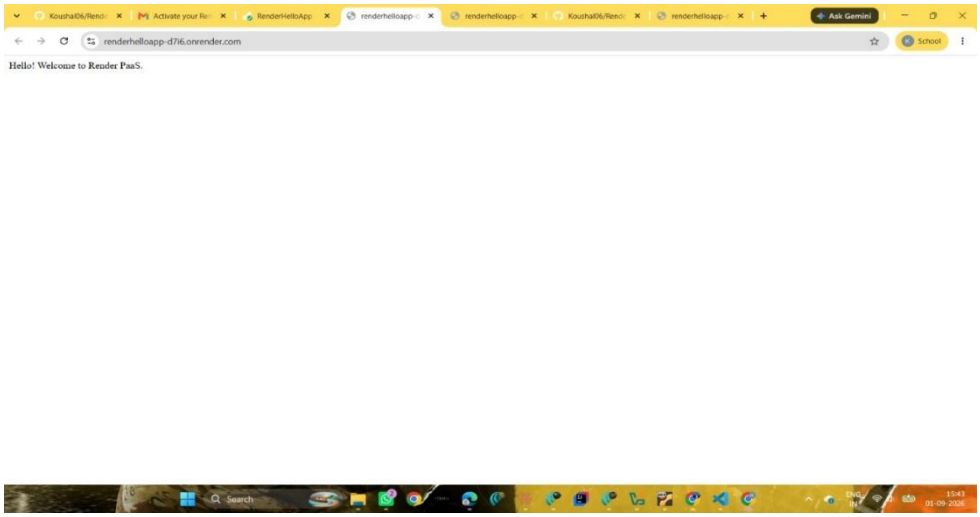



**Fig: Home Page – Hello! Welcome to Render PaaS.**

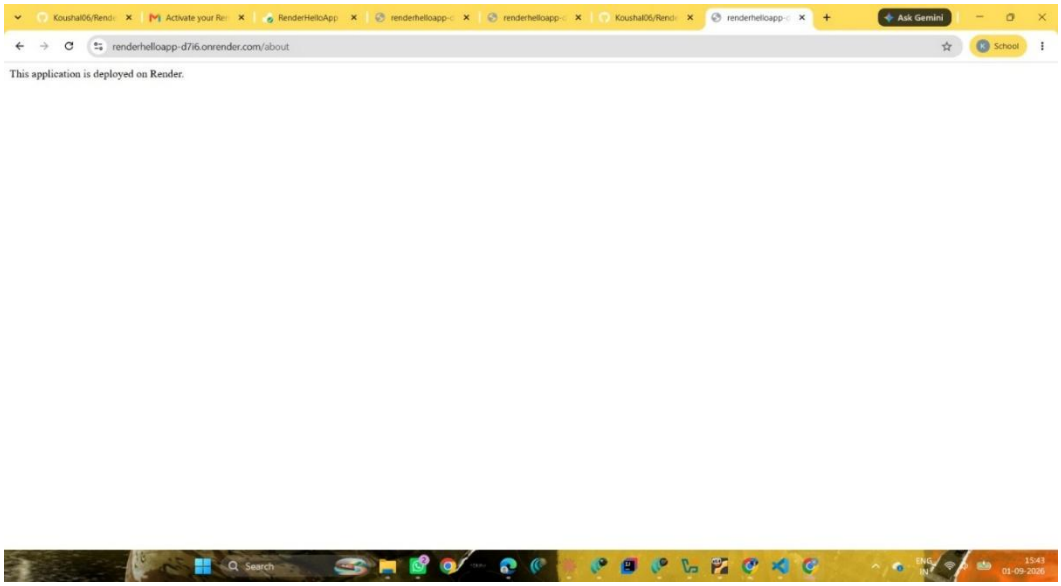



**Fig: About Page – This application is deployed on Render.**

# CONCLUSION

The experiment demonstrated how a PaaS platform simplifies cloud application deployment by providing the runtime environment, dependency installation, web server execution, and public access without requiring manual server management.